Learning Spectral Initializations for Low-Rank Attention

Stanford CS229 Project

Anika Chopra
Department of Mathematics
Stanford University
anika11@stanford.edu

Cody Qiu
Department of Computer Science
Stanford University
cody1212@stanford.edu

Authors' note: This project builds on our CS197 work but addresses a different question. In CS197, we studied fixed low-rank initialization strategies, including PCA, SVD-sqrt, SVD-noscale, SVD-full, and random orthogonal initialization, evaluating them mainly through reconstruction error and spectral diagnostics such as condition number and spectral gap. Here, we introduce data-driven initialization methods, including learned_b, learned_a,b, and an MLP-based singular-value scaling rule. We also shift the evaluation toward predictive objectives: a Koopman proxy based on next-state relative MSE and a downstream next-token experiment using language-model loss and perplexity.

1 Introduction

1.1 Spectral Koopman Attention (SKA)

Attention is highly effective for associative recall, but it requires storing a key-value (KV) cache which grows linearly in sequence length, creating a memory bottleneck for long-context and agentic tasks. **Spectral Koopman Attention** (SKA) addresses this issue by replacing standard attention layers with fixed-size memory modules, enabling constant-memory inference and attention-like retrieval behavior (Sridhar and Johansen, 2026). These SKA modules rely on a compressed key projection $\hat{W}_K$, which maps hidden states into a low-dimensional state space. The construction of this compressed projection will be our primary focus.

In standard attention, each hidden state $h_t \in \mathbb{R}^{d_{\text{model}}}$ is projected into a key vector (Vaswani et al., 2017):

$$k_t = W_K h_t,$$

where $W_K \in \mathbb{R}^{d_k \times d_{\text{model}}}$. SKA instead forms a compressed key representation

$$z_t = \hat{W}_K h_t,$$

where $\hat{W}_K \in \mathbb{R}^{r \times d_{\text{model}}}$ and $r \ll d_k$. Rather than storing all previous key vectors, SKA summarizes the sequence of compressed states using two fixed-size matrices that capture their geometry and temporal evolution:

$$G = \sum_{t=1}^{T} z_t z_t^\top + \epsilon I_r, \qquad M = \sum_{t=1}^{T-1} z_{t+1} z_t^\top.$$

Here, G is a regularized Gram matrix capturing the geometry of the compressed key space, while M captures how compressed key states evolve over time. Since each new token only updates these fixed-size matrices, memory usage is independent of sequence length. SKA assumes compressed states evolve approximately according to a linear dynamical system: $z_{t+1} \approx A z_t$ for some $A \in \mathbb{R}^{r \times r}$. Under this assumption, the Koopman transition operator is obtained by solving the least-squares problem:

$$\arg\min_{A \in \mathbb{R}^{r \times r}} \sum_{t=1}^{T-1} ||z_{t+1} - A z_t||^2,$$

which has closed-form solution:

$$A_w = M G^{-1}.$$

The operator predicts future compressed states using $z_{t+1} \approx A_w z_t$. Hence, the initialization of $\hat{W}_K$ directly affects both the predictive quality of the learned dynamics and the numerical conditioning of the compressed state space. Our project studies how to **initialize** $\hat{W}_K$ from the pretrained key projection matrix W_K. Concretely, we extract W_K from a selected attention layer, collect hidden states from text inputs, and use each strategy to construct a candidate compressed projection $\hat{W}_K$. We first compare low-rank initialization strategies by evaluating **dynamical consistency**, or how well the learned Koopman operator predicts the next compressed key state, and **conditioning**, or how numerically stable the compressed features are. We then test whether these initialization differences matter in a downstream next-token prediction setting by replacing an attention layer with an SKA module and training it using language-model loss.

1.2 Eckart-Young Theorem

The current SKA pipeline uses **random orthogonal initialization** (Sridhar and Johansen, 2026) to initialize the compressed key projection matrix $\hat{W}_K$. This strategy involves randomly generating a matrix of a specified rank r–typically drawn from a Gaussian distribution–and using QR-decomposition to extract the orthogonal component of that random matrix. While this initialization provides a well-conditioned,

unstructured compressed projection, we investigate strategies that exploit the structure of the pretrained key projection matrix W_K when initializing $\hat{W}_K$. We therefore compare random orthogonal initialization with **SVD-based spectral initializations** derived from W_K, as well as **learned spectral transformations** that learn how to rescale the retained singular directions.

SVD provides a natural starting point because it solves the canonical low-rank approximation problem. By the Eckart-Young theorem (Eckart and Young, 1936), for any matrix B with $\mathrm{rank}(B) \leq r$,

$$U_r \Sigma_r V_r^T = \arg \min_{\mathrm{rank}(B) \leq r} \|W_K - B\|_F.$$

Thus, the truncated SVD preserves as much of the pretrained key projection W_K as possible among all rank-r reconstructions. Our learned strategies build on this spectral structure by asking whether the singular-value scaling rule can be improved beyond fixed choices such as SVD-noscale, SVD-sqrt, and SVD-full. We define these strategies in Section 4.

2 Related Work

Efficient attention and long-context models Motivated by the KV-cache bottleneck described above, a broad line of work has explored more memory-efficient alternatives to standard attention. Performer approximates attention with random feature maps (Choromanski et al., 2022), while State Space Models such as Mamba use linear-time state-space dynamics (Gu and Dao, 2024). However, these methods substantially change the retrieval mechanism, which can weaken associative recall in long-context settings. The closest work to ours is Spectral Koopman Attention (SKA), which aims to preserve attention-like recall while avoiding a growing KV cache (Sridhar and Johansen, 2026). Our work builds on SKA by studying how the compressed key projection should be initialized, treating the random orthogonal approach and fixed SVD methods as baselines and exploring learned SVD strategies.

Learned Low-Rank Representations Learned low-rank representations have also been widely studied in language model adaptation. LoRA fine-tunes models through low-rank update matrices, showing that useful model changes can often be captured in a smaller-dimensional subspace (Hu et al., 2021). PiSSA further shows that initializing low-rank factors using SVD structure can improve optimization (Meng et al., 2025). Our work applies this motivation to SKA: rather than using spectral structure to initialize low-rank fine-tuning updates, we use it to initialize the compressed key projection and study whether the spectral scaling rule itself can be learned.

3 Dataset and Features

We use the WikiText-103 corpus introduced by Merity et al. (2017) as our text dataset. WikiText-103 consists of Wikipedia articles and is commonly used for language modeling evaluation. In the Koopman proxy experiments, we use 512 training text sequences, 256 validation sequences, and 256 test sequences. The training split is used to fit the learned spectral transformations, the validation split is used for model selection during development, and the test split is reserved for final evaluation. Each sequence is tokenized using the pretrained Pythia tokenizer and padded or truncated to a maximum length of 256 tokens. Empty text entries are removed before tokenization.

Our input features are hidden states extracted from a pretrained Pythia-160M language model, part of the Pythia model suite introduced by Biderman et al. (2023). For the Koopman proxy experiments, we extract hidden states from Layer 2. For a tokenized sequence of length T, this gives a sequence of feature vectors

$$h_1, \ldots, h_T, \qquad h_t \in \mathbb{R}^{d_{\mathrm{model}}}.$$

Each token position is treated as one discrete time step. Given an initialization $\hat{W}_K$, these hidden-state features are projected into compressed key states $z_t = \hat{W}_K h_t$. Thus, the features used by our learning objective are model-derived rather than manually engineered: the Pythia hidden states h_t, the compressed key states z_t, and spectral features of the pretrained key projection matrix W_K, such as its singular values and singular directions.

For our next-token prediction experiment, we used the same Wikitext corpus and Pythia-160M language model, but evaluate directly on next-token language modeling. In this experiment, a single attention layer (Layer 2) is replaced with an SKA module, and the model is trained over the tokenized Wikitext sequences using LM loss.

4 Methods

4.1 Baseline

Our primary baseline methods are **SVD-noscale, SVD-sqrt, and SVD-full**, which we explain below. Given a pretrained key projection matrix W_K, our goal is to initialize a lower-rank SKA key projection $\hat{W}_K$ that produces stable and useful compressed key representations. In Section 1.2, we mentioned that the best rank-r reconstruction is

$$W_K \approx U_r \Sigma_r V_r^T$$

(Eckart and Young, 1936), where U_r, Σ_r, and V_r^T keep only the top r singular components. However, this reconstruction has shape $d_k \times d_{\text{model}}$, matching the original key projection W_K. In contrast, the compressed SKA key projection must have shape $\hat{W}_K \in \mathbb{R}^{r \times d_{\text{model}}}$. Therefore, we cannot directly use the full reconstruction $U_r \Sigma_r V_r^T$. Instead, we use the SVD components that define an r-dimensional compressed representation:

$$\hat{W}_K = \Sigma_r^b V_r^T.$$

Here, V_r^T selects the dominant input directions of W_K, while the exponent b determines how strongly each direction is scaled. The left singular vectors U_r are not used because they map the compressed coordinates back into the original key-output space, whereas SKA operates directly in the compressed space. Concretely, **SVD-noscale uses** $b = 0$**, SVD-sqrt uses** $b = 1/2$**, and SVD-full uses** $b = 1$. The $b = 0$ strategy is the closest to the random orthogonal baseline, since we consider only the pure orthonormal directions V_r^T with no singular-value weighting. The $b = 1$ choice is a direct application of Eckart-Young, while $b = 1/2$ interpolates between these two choices.

4.2 Learned strategies

To generalize the fixed SVD heuristics, we explored data-driven spectral transformations applied to the truncated singular-value matrix. Instead of using Σ_r directly, we learn a transformation g_θ that determines how each retained singular direction should be weighted in the compressed SKA key projection. Each successive transformation contains more parameters than the previous to allow progressively more flexible scaling rules.

4.2.1 Learned Exponent Scaling (learned_b)

As a simple extension of the fixed SVD baselines, we first considered a one-parameter transformation that learns only the exponent applied to each singular value:

$$g_b(\sigma_i) = \sigma_i^b.$$

This formulation includes our fixed baselines as special cases. For example, $b = \frac{1}{2}$ corresponds to the SVD $\sqrt{\Sigma}$ baseline. Learning b allows the model to adjust the relative weighting of singular directions while keeping the global scale fixed.

4.2.2 Power-Law Spectral Scaling (learned_a,b)

We also considered a more flexible two-parameter power-law transformation:

$$g_{a,b}(\sigma_i) = a\sigma_i^b.$$

Here, b controls the relative weighting of spectral directions, while a controls the overall scale of the compressed projection.

4.2.3 Residual MLP Spectral Scaling

To move beyond fixed power form $a\sigma^b$, we learn a more flexible nonlinear correction with a three-layer MLP with GELU activations. For each singular value σ_i we define

$$\log g_i = 0.5 \log \sigma_i + \alpha f_\theta(\log \sigma_i,\, i/r), \qquad \text{i.e.} \qquad g_i = \sigma_i^{0.5} \exp\big(\alpha f_\theta(\log \sigma_i,\, i/r)\big).$$

We work in log-space because singular values span several orders of magnitude. This normalizes the MLP's inputs, makes f_θ a *multiplicative* correction to the baseline, and guarantees $g_i > 0$. The fixed 0.5 exponent anchors the transformation at the SVD-sqrt initialization, while f_θ learns deviations from this baseline. The inputs $\log \sigma_i$ and i/r represent each direction's magnitude and its relative position in the spectrum, allowing the model to treat leading and trailing singular values differently. The hyperparameter α limits the magnitude of the correction, keeping it near the stable SVD-sqrt baseline. We initialize the final MLP layer to zero so that f_θ initially outputs zero, meaning the model begins exactly at $g_i = \sqrt{\sigma_i}$.

4.2.4 Optimization objective

We trained our strategies on the following loss function:

$$\mathcal{L}(\theta) = \frac{1}{T-1} \sum_{t=1}^{T-1} \|A_w z_t - z_{t+1}\|_2^2 + \lambda \log \kappa(G), \qquad \kappa(G) = \frac{\sigma_{\max}(G)}{\sigma_{\min}(G)}.$$

Here, $\kappa(G)$ is the condition number of the regularized Gram matrix $G = \sum_{t=1}^{T} z_t z_t^\top + \epsilon I_r$. The first term measures Koopman next-state prediction error, while the second penalizes ill-conditioning of the compressed key space. Thus, the objective balances predictive accuracy with numerical stability.

4.2.5 Training and Evaluation

This formulation makes the project a regularized optimization problem over the parameters of a learned spectral transformation g_θ. For the power-law models, these parameters are the scalar coefficients defining the spectral scaling rule. For the residual MLP model, the learnable parameters are the weights and biases of f_θ, while the baseline exponent is fixed at 0.5 and α is treated as a hyperparameter.

We optimized these parameters using gradient-based methods, specifically AdamW, and compared the learned spectral transformations against fixed SVD baselines using transition prediction error, Gram matrix conditioning, and downstream SKA behavior.

To evaluate these models, we measure the following:

1. **Relative mean-squared error**: This is defined as:

$$\text{RelMSE} = \frac{\frac{1}{T-1} \sum_{t=1}^{T-1} \|A_w z_t - z_{t+1}\|_2^2}{\frac{1}{T-1} \sum_{t=1}^{T-1} \|z_{t+1}\|_2^2 + \epsilon}.$$

   where we use $\epsilon = 10^{-12}$ as a small numerical stability constant added to avoid division by zero. This metric measures the prediction error relative to the average squared magnitude of the true next states. Lower relative MSE indicates that the learned Koopman operator more accurately predicts the next compressed key state.

2. **Condition number**: We compute the condition number of the regularized Gram matrix G, defined as:

$$\kappa(G) = \frac{\sigma_{\max}(G)}{\sigma_{\min}(G)}.$$

   This metric captures the numerical stability of the compressed key representation. A high condition number indicates that the compressed states contain nearly redundant or low-energy directions, which can make estimating the Koopman operator unstable. Therefore, a lower condition number indicates a more stable compressed state space.

3. **LM loss/Perplexity**: We evaluate end-to-end language modeling performance using the Hugging Face causal language-modeling loss, computed by passing the input token IDs as labels. This corresponds to the average next-token negative log-likelihood:

$$\mathcal{L}_{LM} = -\frac{1}{T-1}\sum_{t=1}^{T-1} \log p_\theta(w_{t+1} \mid w_1, \ldots, w_t).$$

This measures how well the modified model predicts the next token after replacing the original key projection with an SKA initialization. We also report perplexity,

$$\mathrm{PPL} = \exp(\mathcal{L}_{LM}),$$

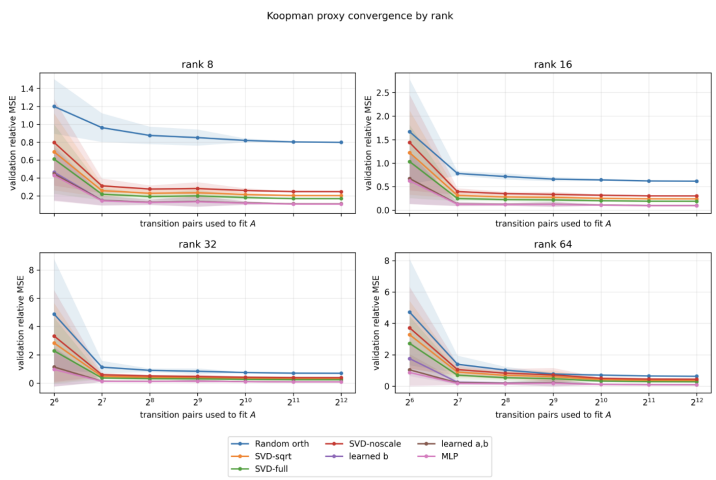
where lower LM loss and lower perplexity indicate better next-token prediction performance.

5 Experiments / Results / Discussion

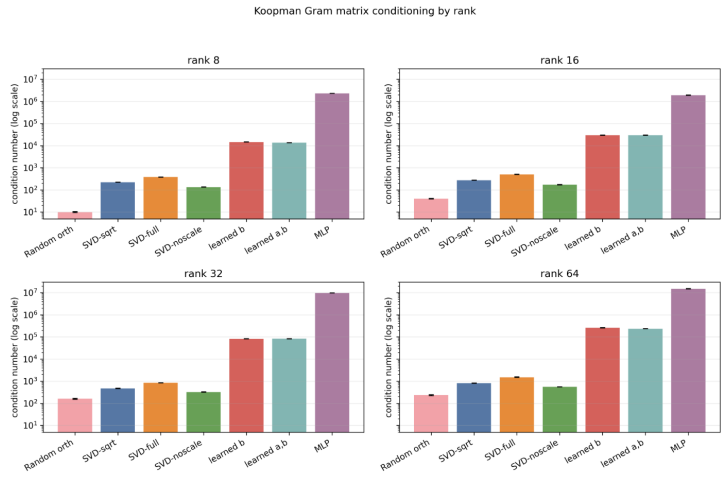
For our first experiment, the **Koopman proxy experiment**, our goals are twofold. First, because full next-token training is computationally expensive, we use the proxy objective as a model-selection stage for the learned initialization strategies, selecting hyperparameters before running the downstream language-modeling experiment. Second, we evaluate how well each initialization supports Koopman prediction, measured by how accurately $A_w z_t$ predicts z_{t+1}.

We sample 512 training sequences, 256 validation sequences, and 256 test sequences from WikiText-103. Each sequence is tokenized with the Pythia tokenizer and preprocessed as described in Section 3. For each strategy, we compute relative mean-squared error for different numbers of transition pairs, as well as the condition number of the corresponding Gram matrix. For learned_a,b and learned_b, we used AdamW with a learning rate of 0.01 and trained over 1000 steps. For the MLP strategy, our learning rate was 0.001.

We use 5-fold cross validation to choose the hyperparameters λ and α. For each setting, we train on four folds and evaluate relative Koopman prediction error on the held-out fold, selecting the setting with the lowest average validation error. We tune $\lambda \in [0, 10^{-5}, 10^{-4}, 10^{-3}, 10^{-2}]$ for learned_b and learned_a,b, and tune both λ and $\alpha \in [0, 0.03, 0.1, 0.3]$ for the MLP. We repeat this procedure across ranks $8, 16, 32, 64$, averaging the results over 5 random seeds. The results are displayed below.



(a) Koopman proxy convergence



(b) Gram matrix conditioning

Figure 1: Comparison of initialization strategies using Koopman prediction and conditioning metrics.

Cross validation selected $\lambda = 0$ for all learned strategies and $\alpha = 0.3$ for the MLP. This suggests that in the proxy experiment, relative MSE is driven primarily by transition-prediction accuracy rather than Gram matrix conditioning. Figure 1a shows that most strategies plateau once about 2^7 transition pairs are used to fit A_w, suggesting that additional transition pairs provide limited benefit for this proxy task.

Across all ranks, the learned strategies generally achieve the lowest relative MSE in the Koopman proxy experiment, with the MLP performing strongest. Random orthogonal performs worst on this metric, while the fixed SVD strategies fall in between. However, Figure 1b shows the opposite trend for conditioning: the learned strategies have much higher condition numbers, while random orthogonal is the best conditioned across all ranks. Thus, learned spectral scaling improves Koopman next-state prediction in the proxy task, but at the cost of worse numerical conditioning. One possible explanation is that random orthogonal initialization is stable but unstructured. Since it does not use information from the pretrained key projection matrix W_K, it may project hidden states into a subspace that discards attention-relevant structure, making z_{t+1} harder to predict from z_t. In contrast, the SVD-based and learned spectral strategies use directions derived from W_K, which may better preserve structure useful for Koopman prediction. We also observe that relative MSE generally decreases as rank increases, which is expected: higher-rank compressed projections retain more information, giving the Koopman operator a richer representation from which to predict the next compressed state.

Also, we do not see strong evidence that the learned spectral strategies overfit to the proxy training set: the methods were selected by cross-validation and continued to perform well on held-out proxy evaluations. However, a strong Koopman predictor may not capture all

of the interactions involved in actual language-model training, and so we next evaluate whether these proxy improvements transfer to a more realistic downstream setting. Because the Koopman proxy experiment is much cheaper than full next-token training, we use it as a model-selection stage for the learned spectral strategies, reusing the learned scaling parameters in the **Next-Token experiment**. In this final experiment, we replace Layer 2 of Pythia-160M with an SKA module initialized using each strategy and rank. We freeze the rest of the language model and train only the SKA-related parameters on tokenized WikiText-103 sequences using the standard next-token language-modeling objective. We train for 300 gradient steps with a gradient accumulation of 4 and batch size of 4 and evaluate the final models on held-out WikiText sequences using LM loss and perplexity. The results, averaged over six random seeds, are shown in Figure 2 and Table 1.

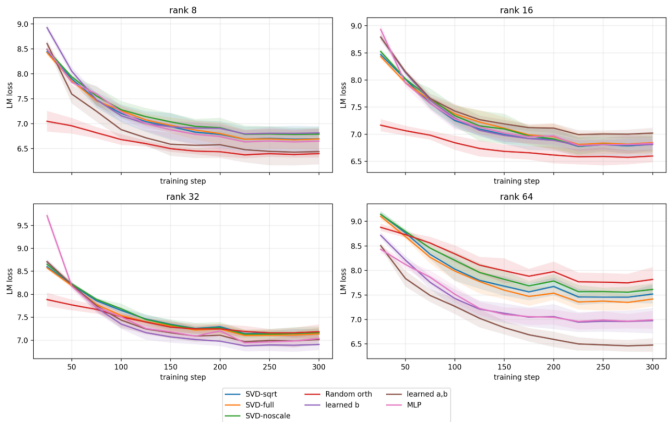


Figure 2: Next-token training curves across initialization strategies.

Rank 8			Rank 16		
Strategy	**Loss**	**PPL**	**Strategy**	**Loss**	**PPL**
Random orth	6.092	446.6	**Random orth**	6.213	513.6
learned_a,b	6.287	585.8	SVD-noscale	6.548	706.5
MLP	6.456	650.9	learned_b	6.575	721.9
SVD-noscale	6.602	740.5	MLP	6.586	732.3
SVD-full	6.609	749.5	SVD-sqrt	6.645	782.1
SVD-sqrt	6.687	833.7	SVD-full	6.677	808.9
learned_b	6.780	885.4	learned_a,b	6.767	870.1

Rank 32			Rank 64		
Strategy	**Loss**	**PPL**	**Strategy**	**Loss**	**PPL**
learned_b	6.782	887.0	**learned_a,b**	6.466	655.6
learned_a,b	6.809	912.7	learned_b	6.860	966.9
MLP	6.831	931.1	MLP	7.065	1177.7
Random orth	6.879	986.4	SVD-sqrt	7.124	1261.5
SVD-noscale	6.892	997.0	Random orth	7.181	1340.4
SVD-full	6.920	1017.2	SVD-full	7.191	1351.2
SVD-sqrt	6.966	1068.7	SVD-noscale	7.209	1359.2

Table 1: Final next-token performance after 300 training steps, averaged over six random seeds. Lower loss and perplexity are better.

Overall, the downstream results are less uniform than the Koopman proxy results. Figure 2 shows that most strategies largely converge by steps 200–300. Table 1 shows that final LM loss and perplexity are strongly rank-dependent. Random orthogonal performs best at lower ranks, while learned spectral strategies generally perform better at higher ranks. Paired t-tests with significance defined as $p < 0.05$ show that random orthogonal is highly competitive, performing significantly better than most methods at rank 8 and all methods at rank 16. At rank 32, the differences are less decisive: learned strategies occupy the top positions, but not all pairwise differences are statistically significant. At rank 64, learned strategies such as learned_b and learned_a,b significantly outperform random orthogonal and the fixed SVD baselines, but the difference between any two learned strategies are insignificant.

One possible explanation is that, at smaller ranks, the SKA module is severely bottlenecked and must compress the original key space into very few dimensions. In this regime, all methods discard substantial information, so alignment with the pretrained key projection W_K may provide limited benefit. Random orthogonal may instead work well because it produces a stable, well-conditioned compressed representation that is not dominated by a small number of spectral directions. Fixed SVD methods use directions from W_K, but their hand-designed singular-value scalings may be too rigid: at low ranks they can overemphasize a few dominant modes, while at high ranks they do not adapt the spectrum as effectively as the learned strategies. As rank increases, the compressed space can preserve more of the pretrained key geometry, making alignment with W_K more useful. This may explain why learned spectral strategies become stronger at higher ranks despite their worse conditioning.

These results also show that improvements on the Koopman proxy objective do not transfer uniformly to end-to-end language modeling performance. The proxy objective measures whether compressed key states follow a predictable linear transition, while LM loss depends on how the full retrofitted attention layer interacts with the frozen language model. Thus, the Koopman proxy is useful for selecting promising initialization strategies, but it is not a complete substitute for downstream next-token evaluation.

6 Conclusion / Future Work

In this project, we studied whether learned spectral initialization rules can improve the compressed key projections used in SKA. In the proxy experiment, learned strategies–particularly the MLP–achieved the lowest MSE across ranks, indicating how data-driven methods can produce compressed states that are easier for the Koopman operator to predict. These strategies, however, had worse conditioning than the fixed SVD and random orthogonal strategies, indicating a tradeoff between predictive quality and numerical stability.

In the next-token prediction experiment, the results were more rank-dependent: random orthogonal was strongest at smaller ranks, while learned spectral strategies became more effective at larger ranks. Overall, our results suggest that learned spectral initialization is most useful when the compressed representation has enough capacity to exploit pretrained key-matrix structure.

Our experiments also have several limitations: we evaluated on a relatively small Pythia-160M model and replaced only one attention layer, even though different transformer layers may use attention in qualitatively different ways. With more time and compute, we would test larger models, replace multiple layers, and study whether the best initialization strategy changes with layer depth. We would also train the learned scaling rules directly with language-model loss rather than selecting them through the cheaper Koopman proxy objective, which may better align the initialization with downstream performance.

7 Contributions

- **Anika**: Ideated and designed data-driven strategies g_θ, formulated loss function, implemented MLP and proxy experiment.
- **Cody**: Formulated loss function, conducted cross-validation for hyperparameters, implemented proxy and next-token experiment.

References

Stella Biderman, Hailey Schoelkopf, Quentin Anthony, Herbie Bradley, Kyle O'Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, Aviya Skowron, Lintang Sutawika, and Oskar van der Wal. 2023. Pythia: A suite for analyzing large language models across training and scaling. In *International Conference on Machine Learning*.

Krzysztof Choromanski, Valerii Likhosherstov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Davis, Afroz Mohiuddin, Lukasz Kaiser, David Belanger, Lucy Colwell, and Adrian Weller. 2022. Rethinking attention with performers.

Carl Eckart and Gale Young. 1936. The approximation of one matrix by another of lower rank. *Psychometrika*, 1(3):211–218.

Albert Gu and Tri Dao. 2024. Mamba: Linear-time sequence modeling with selective state spaces.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, and Weizhu Chen. 2021. Lora: Low-rank adaptation of large language models. *CoRR*, abs/2106.09685.

Fanxu Meng, Zhaohui Wang, and Muhan Zhang. 2025. Pissa: Principal singular values and singular vectors adaptation of large language models.

Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2017. Pointer sentinel mixture models. In *ICLR*.

Anupama Sridhar and Alexander Johansen. 2026. Echo: Kv-cache-free associative recall with spectral koopman operators.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *CoRR*, abs/1706.03762.